

grounding_agent — Evaluation Framework

Vineet Singh

May 13, 2026

CONTENTS

grounding_agent — presentation	
In one sentence	
What we built	
Seven-category failure taxonomy	
Contract generated from policy	
Six judges	
Runner + evaluator + comparator	
Structured event log	
120 tests	
How we built it	
Day 1 (build the pipeline)	
Day 2 (run + compare + write)	
Forensics-driven iteration (no shortcuts)	
Key numbers (iter-3 final state)	
Variant overview	
Per-dimension pass rate (iter-3 v0, all-split)	
Judge cost decomposition	
What the framework caught that vibes-eval would not	
What's missing / known limitations	

Way forward (in order of leverage)

Cost + ops at production scale

What we learned (the meta-points)

Where everything lives

grounding_agent — presentation

Submission for Meraki Labs Founding AI Engineer Work Trial, PS4 — *Evaluation Framework from Scratch*. Two-day work-trial, scope ruthless. Read time $\approx$ 5 minutes.

In one sentence

A multi-dimensional evaluation framework for tool-calling LLM agents, applied to τ -bench's airline customer-support agent, where the **disagreements between auto-eval and τ -bench's programmatic reward are the result** — and where structured forensic iteration turned the framework's own weaknesses into measured improvements.

What we built

SEVEN-CATEGORY FAILURE TAXONOMY

A frozen taxonomy of structural failure modes for tool-using agents. Each category is grounded in a load-bearing policy clause.

Category	Judge kind	What it catches
<code>policy_compliance</code>	semantic	Business rules ignored or applied wrong
<code>confirmation_discipline</code>	deterministic	Mutating tool call without preceding user "yes"
<code>information_grounding</code>	semantic	Facts not in tool outputs / user messages
<code>scope_adherence</code>	semantic	Mis-applied transfer-to-human decision
<code>tool_sequence_correctness</code>	deterministic	Mutation without prerequisite read
<code>tool_argument_correctness</code>	deterministic	Tool call returned <code>Error: ...</code>
<code>task_completion</code>	observed via τ -bench reward	End-to-end goal achievement

3 semantic + 3 deterministic + 1 reward-observed = **7 dimensions, 6 judges**.

CONTRACT GENERATED FROM POLICY

One LLM call against `vendor/tau_bench_airline/policy.md` produces `data/contract.json` — obligations + forbidden behaviors + tool_sequences, each tagged to a taxonomy category. Validator gates save and load. **No hand-curation** — removes the “candidate-curated favourable obligations” critique.

SIX JUDGES

- 3 semantic LLM judges (`policy_compliance`, `information_grounding`, `scope_adherence`). Each receives an “AGENT ACTIONS” block first, then the full trajectory for context, with explicit ground rules.
- 3 deterministic Python checks (`confirmation_discipline`, `tool_sequence_correctness`, `tool_argument_correctness`). Each walks the trajectory once and emits a binary verdict + a continuous `score` field where applicable.

RUNNER + EVALUATOR + COMPARATOR

- `runner.run_task` drives the τ -bench tool-calling agent, captures the trajectory, classifies termination (`completed` / `transfer` / `max_steps`), extracts tool errors.
- `evaluator.evaluate_trajectory` applies all six judges.
- `compare.py` produces confusion matrices, per-clause citation counts, termination distributions, and reward-kind decomposition (`r_actions` vs `r_outputs` vs `no_grade`).

STRUCTURED EVENT LOG

JSON-Lines at `results/logs/<run_id>/<variant>.jsonl`. Captures run-level / task-level / per-judge events with timing and verdicts. Replayable. Made forensics pass 2 reproducible.

120 TESTS

Every module covered. Test fixtures use the exact OpenAI chat- completion message shape `tau_bench.agents.tool_calling_agent.solve` emits — no toy data.

How we built it

DAY 1 (BUILD THE PIPELINE)

Scaffold → taxonomy → contract → judges → runner → evaluator → smoke test on 2 tasks. Six modules, all under 300 lines. Per-chunk code reviews in `code_review/`.

DAY 2 (RUN + COMPARE + WRITE)

Full 20x2 evaluation. v2 prompt variant = v0 + “execution discipline” preamble. First forensics report → six load-bearing findings.

FORENSICS-DRIVEN ITERATION (NO SHORTCUTS)

Forensics pass 1 (`results/forensics.md`) mined the trajectories for what the WRITEUP missed. Six findings bucketed into four issue classes:

- **Bucket A** — `confirmation_discipline` LLM judge was provably worse than a 10-line Python heuristic.
- **Bucket B** — semantic judges anchored on user *requests*, not agent *actions*. Verified on a canonical refuse-and-transfer case.
- **Bucket C** — missing dimension: arithmetic errors dominated failures and the taxonomy had no `tool_argument_correctness`.
- **Bucket D** — reward kind (`r_actions` vs `r_outputs`) and termination kind (`max_steps` etc.) not surfaced.

Iteration 2 — fixed all four buckets + added structured event logging.

Forensics pass 2 (`results/forensics_v2.md`) verified each bucket landed its effect:

- Bucket A: confirmation pass-rate 0% → 65% (v0). FN count 3 → 0.
- Bucket B: task 15 (canonical refuse case) went 0/5 PASS → 6/6 PASS.
- Bucket C: 18 tool errors caught (v0), 39 caught (v2).
- Bucket D: every metric now surfaced in compare output.

Identified one residual issue: contract generator still mistagging `fb-modify-basic-economy` and 3 other business-rule prohibitions.

Iteration 3 — added CATEGORY DEFINITIONS + DECISION HEURISTIC to the contract-generation prompt. Re-judged cached trajectories via a new `--judge-only` flag (no agent re-rollout).

Forensics pass 3 (`results/forensics_v3.md`) measured iter-3 deltas: three target mistags moved correctly; `fb-modify-basic- economy` citations dropped 9 → 5. One new mistag emerged. Stopped at diminishing returns.

Key numbers (iter-3 final state)

VARIANT OVERVIEW

metric	v0 (wiki as-is)	v2 (discipline preamble)
τ-bench reward pass	10% (2/20)	10% (2/20)
avg messages / run	33.9	33.1
total cost	\$0.14	\$0.15
completed	14	14
transfer	3	2
max_steps	3	4
tool errors (<code>book_reservation</code>)	16	34

PER-DIMENSION PASS RATE (ITER-3 V0, ALL-SPLIT)

dimension	iter-1	iter-3	move
<code>confirmation_discipline</code>	0%	65%	+65pp  deterministic
<code>information_grounding</code>	37%	70%	+33pp  Bucket B
<code>scope_adherence</code>	5%	30%	+25pp  Bucket B
<code>policy_compliance</code>	11%	20%	+9pp
<code>tool_sequence_correctness</code>	74%	75%	unchanged
<code>tool_argument_correctness</code>	—	65%	new  Bucket C

JUDGE COST DECOMPOSITION

dimension	mean duration	kind
policy_compliance	2 900 ms	semantic
scope_adherence	2 700 ms	semantic
information_grounding	1 900 ms	semantic
All deterministic judges	< 1 ms	deterministic

Per trajectory: 7-8 s of semantic judge time. At 100k/day this is the dominant cost line (\$15/day in tokens).

What the framework caught that vibes-eval would not

- v2 made the agent worse on reward, not better** (iter-1: 16% → 5%). The “execution discipline” preamble added confirmation rounds, lengthened trajectories ~30%, and hit `max_steps`. Multi- dimensional eval located the cause precisely; single-score eval would have stopped at “v2 is worse.”
- confirmation_discipline LLM judge was systemically wrong.** A 10-line Python heuristic out-performed it. The framework caught its own weakness because the per-dimension verdicts were debugger-friendly.
- Semantic judges anchored on conversation subject, not agent action.** Task 15 (refuse-and-transfer) was the canonical case; Bucket B’s “AGENT ACTIONS” block fixed it.
- The real failure mode is arithmetic.** Payment-split mismatches, gift-card balance shortfalls, invented user_ids. The original six-category taxonomy didn’t have a dimension for this; Bucket C added one.
- Generated contracts mistag clauses.** Three iterations of prompt-engineering reduced — but did not eliminate — the noise. The framework’s diagnostic output exposed the mistag at clause-id granularity, making the fix tractable.

What's missing / known limitations

gap	why we accepted it	how to fix
No coverage for argument-choice correctness (right flight, right cabin)	The deterministic checker catches <i>invalid</i> args (tool returns Error); it doesn't catch <i>suboptimal</i> args the tool accepts	Add a semantic judge that compares the agent's chosen args against tool returns + user constraints
<code>r_outputs</code> tasks (text-match grading) never pass with gpt-4o-mini	Our taxonomy has no "answer quality" dimension; these are 2-4 tasks per variant	Add a <code>response_quality</code> judge that scores the final assistant message against the user's question
Contract mistagging is structural noise of LLM generation	We accepted it to preserve "no curated obligations"	In production: one human-in-the-loop tag review after generation (validator infra already there)
Single trial per task (n=20)	Multi-trial would have multiplied cost in a 2-day window	Multi-trial at higher N for statistical confidence on v0/v2 deltas
Synchronous judges	Sufficient at n=40	<code>evaluate_trajectory</code> is pure; async-ify in 30 min for production
One model (gpt-4o-mini) for agent + user-sim + judges	Cost discipline	Tier: gpt-4o-mini for sampled inspection, gpt-4o for arbitration on flagged trajectories
No web UI / dashboard	Out of scope	Markdown + JSON outputs suffice for review

Way forward (in order of leverage)

1. `tool_argument_choice_correctness` — close the coverage gap that produces most remaining FPs. Semantic judge that re-derives the *correct* args from tool returns + user constraints and compares to the agent's call. Highest leverage: catches the modal failure mode (wrong flight numbers, wrong payment splits) that no current dimension covers.
2. `response_quality` dimension for `r_outputs` tasks. Currently 0% pass; framework can't say why.

3. **Tag-review step** in `scripts/generate_contract.py`: emit the draft contract, render it diff-friendly, accept a tag-only overlay file. One-pass human review yields a clean tagged contract while keeping clause text generator-authored.
 4. **Multi-trial at $N \geq 50$** for tighter v0/v2 deltas. The iter-2 $\rightarrow$ iter-3 $\rightarrow$ iter-1 reward variance (16% $\rightarrow$ 10% $\rightarrow$ 5% across runs) is pure stochasticity at this sample size.
 5. **Async judges** so end-to-end wall time isn't bottlenecked by sequential litellm calls.
 6. **Tier production judges** by stakes — deterministic on 100%, semantic on a sampled %, gpt-4o arbitration on flagged.
 7. **Active prompt-tuning of the semantic judges.** Forensics pass 2 showed `policy_compliance` and `information_grounding` still over-cite. Few-shot examples in the system prompt would help.
 8. **Multi-agent comparison.** Right now the framework evaluates one agent (τ -bench customer-support). Plumbing for “evaluate any tool-calling agent against a policy” is one indirection.
-

Cost + ops at production scale

- **Per-trajectory cost** with current setup: $\sim \$0.015$ (agent + user-sim)
 - 3 semantic judges + 3 free deterministic). At 100k/day = $\sim \$1.5k/day$.
 - **Reducing cost without losing signal:**
 - Run deterministic judges (free, ~ 0 ms each) on 100%.
 - Sample semantic judges to 1-5%.
 - Reserve gpt-4o for arbitration on flagged trajectories.
 - **Reliability:** errors recorded per-task, don't crash the run. Idempotent caching means a partial run picks up cleanly. Validator infra prevents malformed contracts from being silently loaded.
 - **Observability:** every run produces a JSON-Lines event log
 - per-task results JSON + a rendered comparison markdown. All three are mineable post-hoc.
-

What we learned (the meta-points)

1. **Multi-dimensional eval's value emerges in the disagreements with ground truth, not in the agreements.** If our framework matched the reward perfectly, it would be redundant; it earns its keep when it shows you *why* the reward failed.

2. **LLM judges and deterministic judges are not interchangeable.** Confirmation is mechanically observable; an LLM was strictly worse. Tool argument validity is mechanically observable; an LLM is unnecessary. Reserve LLM judges for genuinely subjective dimensions.
3. **Forensics-driven iteration is the right workflow.** Build → observe → bucketize → fix → re-observe. Each iteration's data is preserved so the trajectory of improvement is auditable.
4. **The framework's output is debuggable because it cites clause-ids.** Pass-1 forensics couldn't have identified `obl-confirm-action` as the always-cited clause without structured citations. Same for the mistagged clauses.
5. **Generated contracts have structural noise that bounds eval precision.** No amount of prompt engineering eliminates it entirely. Production deployment needs human-in-the-loop on the tag layer; the validator infrastructure is the natural home for that review.

Where everything lives

artifact	path
Code	<code>grounding_agent/</code> (7 modules, all ≤ 300 lines)
Tests	<code>tests/</code> (120 passing)
Contract	<code>data/contract.json</code> (+ <code>.iter2.json</code> backup)
Event logs	<code>results/logs/<run_id>/<variant>.jsonl</code>
Forensics docs	<code>results/forensics.md</code> , <code>forensics_v2.md</code> , <code>forensics_v3.md</code>
Comparison reports	<code>results/comparison.md</code> (+ <code>.iter2.md</code> , <code>.pre.md</code>)
Per-iteration results	<code>results/{v0,v2}_results.json</code> + <code>.iter2.json</code> + <code>.pre.json</code>
Code reviews	<code>code_review/</code> (per-chunk dated reviews)
Session log	<code>knowledge.md</code>
Error log	<code>errors.md</code>
Project brief	<code>BRIEF.md</code>
Long-form writeup	<code>WRITEUP.md</code>
This file	<code>PRESENTATION.md</code>
Repo entry point	<code>README.md</code>